Report Oct. 29, 2026

AI can turn security fixes into working exploits before users receive an update

In our experiments, an open-weight model reliably exploited the "patch gap" to compromise six recent Firefox releases, costing about $1,000 each to carry out



By Aiden Ament, Yafah Edelman, Alexander Barry, Timothée Chauvin, Lee Bousfield, and Jaeho Lee

Overview

GLM 5.3, a widely available open-weight model released by Z.ai in August, can identify and exploit vulnerabilities in recent versions of the Mozilla Firefox browser using security fixes that have been published but not yet shipped to users. The window between a fix being published and reaching users is known as the *patch gap*.

In our experiments, GLM 5.3 used this gap to build a webpage that, when visited by a Firefox user, read a file from the visitor's computer and sent it to the attacker with no further user interaction. In particular, we found that any Firefox version released between September 17 and October 26 could be compromised in this way using only information available at the time of release.

The patch gap is a known weakness in software release practices, and it exists widely across software, e.g., browsers, phone operating systems, and router firmware. Exploiting it previously required significant time and expertise, or special access to frontier models with advanced cyber capabilities, so few attackers could target these vulnerabilities before they were fixed.

Our central finding is that widely available open-weight models can now do much, or even all, of this work on their own. We deliberately designed our setup to emulate what a relatively unsophisticated actor with limited resources could achieve, using only

information available at the time of each version's release. Once set up, it cost only about $1,000 to successfully attack each Firefox version.

In all of our compromises of recent Firefox releases, GLM 5.3 also found and used an unpublished vulnerability in conjunction with those found via the patch gap. We contacted the Mozilla team to disclose our findings and discuss ways to mitigate these attacks. Mozilla has since shipped a new version of Firefox (159.0, released on October 27, 2026) that patches the previously undisclosed vulnerability, which should make it more difficult for GLM 5.3 to compromise Firefox via patch-gap exploits.

Open-source projects are particularly vulnerable to this kind of attack as their code is publicly available, but we suspect any software project where a model can find the content of upcoming security changes before release is vulnerable, including closed-source projects.[1] We anticipate this threat vector will worsen as open-weight cyber capabilities continue to improve.

Further findings from our experiments on Firefox:

- **GLM 5.3 exploited the patch gap despite existing countermeasures.** To slow down would-be attackers, published security fixes are obfuscated as ordinary improvements and do not disclose details of underlying vulnerabilities. GLM 5.3 was able to read the code and determine which changes related to security patches.
- **The type of exploit we demonstrated can be extremely serious.** We used GLM 5.3 to create a website that, simply when visited by a Firefox user and without further interaction, gave the attacker the ability to exfiltrate files from the user's computer. Firefox has over 200 million active users, and this exploit could be used to read their passwords stored in the browser, steal authentication cookies, or view other private information on the user's computer.
- **We did not trigger any safeguards or refusals from any provider, including Z.ai, when using GLM 5.3 in this way.** We framed the task as a security exercise, but our actions were otherwise indistinguishable from those of a real attacker.
- **Similar attacks on Google Chrome have not succeeded as of the time of writing.** We do not take this as strong evidence that attacks could not succeed with more

advanced elicitation or by using more capable open-weight models (potentially including the recently announced Kimi K3.1 Pro).

Background

In April 2026, Anthropic's announcement of Claude Mythos Preview marked a leap in AI cybersecurity capabilities. Since then, it has been clear that frontier AI models are capable of detecting vulnerabilities and crafting complex exploits in real production software. Anthropic recently demonstrated this by giving examples of how an internal model was able to find and exploit critical vulnerabilities in widely used software.

Frontier AI labs have responded by giving some defenders early access to advanced capabilities (via projects like Glasswing and Daybreak) and heavily restricting broader access. This strategy is meant to allow defenders to identify and patch vulnerabilities before attackers can exploit them. However, a gap often exists between a vulnerability fix becoming public and it being deployed to all the users of the relevant software. This *patch gap* exists in two places: fixes becoming public before they are released to users, and users continuing to run old software after an update. In our research, we focus on the gap between fixes becoming public and their general release.

Beta versions of software, which developers use to test code and find newly introduced bugs, are one mechanism by which vulnerability fixes become public before they are generally released. Betas typically have a smaller user base than public releases, made up of users willing to test newer and potentially unstable features. For open-source projects, information is even more broadly available, as the entire codebase is public and upcoming changes are typically viewable in beta or "nightly" code branches.

The patch gap is a known security issue, but the time and expertise historically required to exploit it have limited how many actors can do so (and the naturally limited lifetime of exploits developed in this way means attackers may choose to spend their attention elsewhere). Advancing AI cyber capabilities have changed this calculus.

Although unrestricted access (without cyber guardrails) to the most capable frontier models is tightly controlled, open-weight models are broadly available to anyone without

restriction.[2] This open access can benefit cyber defenders by enabling preemptive vulnerability detection and incident response, especially for defenders without access to restricted frontier models. While we focus here on an offensive capability, we do not make claims on how open-weight models will impact the balance between offense and defense.

Open-weight models have historically been unable to reliably exploit the patch gap, but their capabilities are steadily improving. To see whether that limitation still holds, we tested whether GLM 5.3, a widely available open-weight model released in August 2026, could use the patch gap to compromise the most recent releases of Firefox, a web browser with over 200 million active users.

Methodology

We tasked GLM 5.3 with creating a webpage that, when visited by a Firefox user, reads a local file on their system.[3] This can be used to read the user's private documents, copy passwords stored in Firefox, and steal authentication cookies, allowing the attacker to log into some online services as the user. We chose this target because it reflects an attack severe enough to cause real damage to users.

We provided the model with the specific Firefox version on the user's computer and some basic tools, then let it continue working until it developed a successful exploit. To avoid impacting third parties, we did not give the model internet access; instead, we directly provided the publicly available information we thought it would likely need. We provided only information available at the time of the target versions' release, so the model could not use information about vulnerabilities discovered later.

We designed the environment to roughly approximate what an attacker with minimal technical experience and standard development tools could elicit from the model if they let it run online. We accessed GLM 5.3 through both its first-party API and third-party APIs, framing the task as a security exercise. Despite our actions being indistinguishable from those of someone attempting to target real users, at no point was our model use restricted, and we encountered no refusals.

Beyond basic instructions to target the patch gap (i.e., inspect bug fixes not yet incorporated into the stable release), we let the model run autonomously with no user input. The model identified upcoming security fixes, selected vulnerabilities, and built a complex exploit chain on its own.

Results

GLM 5.3 successfully developed attacks against all six Firefox releases between September 17 and October 26,[4] using only information available at the time of each version's release. Visiting the webpage built by the model caused the browser to read a local file and send its contents to the attacker's server without further user action.[5]

The attacks took substantial effort for the model to develop. In one attempt, GLM 5.3 used an upcoming Firefox fix to identify a memory safety flaw, then developed a way to read and modify memory inside the browser, but Firefox's security protections still prevented the model from reading the target file. The model's final attack composed the initial exploit with multiple additional stages to circumvent the remaining layers of security. The model diagnosed crashes, abandoned unsuccessful approaches, and iterated its approach until simply visiting the website was enough to access the file. We gave GLM 5.3 up to 48 hours to find an exploit, but its fastest attempt needed only 12 hours.

In addition to vulnerabilities discovered through the patch gap, our attacks on the most recent Firefox versions also relied on an unpublished vulnerability that the model rediscovered on each attempt. This allowed it to leverage limited control of the browser into the ability to read files from the computer.[6] We shared this vulnerability with Mozilla, and it is now patched as of the October 27 Firefox 159.0 release. We have not yet confirmed whether similar attacks can succeed on 159.0 given this patch, but suspect it should make things at least substantially more difficult.

Our total research spend targeting Firefox exceeded $100,000 in API costs, including experimentation to build a realistic offline environment and our preliminary testing. However, once we developed our harness and pointed GLM 5.3 in the right direction, exploiting the browser became cheap and reliable. Our marginal cost of compromising a

new version of Firefox was about $1,000 at API pricing, and would potentially have been lower if using subscription accounts.

An attacker unconcerned with the risks of letting an autonomous agent develop an exploit with internet access could replicate our setup much more cheaply and easily by giving the model unrestricted internet access instead of going through the effort of running it offline in a sandbox.

After we discovered GLM 5.3's ability to compromise Firefox, we decided to assess whether a similar setup could compromise Google Chrome. We used the same harness and techniques and, as of the time of writing, have not been able to create an equivalent exploit, despite spending about $100,000 in API costs on attempts.

Kimi K3.1 Pro was released via first-party API access on October 23, with the weights planned to be released in early November. Kimi K3.1 Pro has not been part of our experiments, but limited benchmark data suggests it may outperform GLM 5.3 on offensive cyber tasks. We plan on testing more advanced open models on a wider variety of software in the future.

Implications for other software

The conditions needed for patch-gap attacks are common throughout the software industry. Whenever a security fix becomes visible before it reaches users, an attacker has the opportunity to reverse engineer it into an exploit. In September 2026, we saw patch-gap exploits targeting Chrome. In the same month, a vendor published patched builds of PHP more than six hours ahead of the coordinated release of patches for two critical vulnerabilities. These early patches could have contributed to thousands of exploit attempts before the public release (which then climbed to millions immediately afterward).

While Firefox and Chromium are open source, the patch gap also poses risks to closed-source software. Anthropic demonstrated that Mythos is capable of exploiting the patch gap on Windows without any of the project's code publicly available. We did not explicitly test the ability of GLM 5.3 to exploit the patch gap on closed-source projects, but as

open-weight cyber capabilities continue to improve, it will plausibly be possible soon, if it is not already. We are especially concerned about software that users update less frequently, like router firmware or phone operating systems, because widespread compromise can have serious consequences and most users have a much longer window of vulnerability.

Addressing this threat will likely require reducing the time that vulnerability fixes remain public before being incorporated into the general release. We created exploits within two days, but other attackers could be faster. Merely providing defenders access to stronger closed models does not solve the underlying structural issue; a public fix, regardless of how it is found, inherently flags where vulnerabilities are. Defenders have a few levers to pull to mitigate these attacks, such as keeping security fixes private until they are ready to ship or shipping them from beta to release more quickly. Both Firefox and Google Chrome have begun shipping updates every week. Regardless of the approach, it should be assumed that once a vulnerability fix is in published code, attackers can and will use it to target the current release. Defenders should avoid patching multiple vulnerabilities simultaneously if they can be chained together.

Conclusion

GLM 5.3 compromised six of the most recent Firefox releases by exploiting the patch gap. We used a deliberately basic setup, and despite this, the model reliably built successful exploits without human input, at a marginal cost of roughly a thousand dollars per Firefox version compromised. These attacks relied on a key non-public vulnerability that has since been patched, and we were not able to achieve similar outcomes on Chrome (but we do not think this rules out similar attacks succeeding with better elicitation or stronger open-weight models).

We see these results as evidence of the diffusion of advanced cyber capabilities; many other pieces of software are likely already vulnerable to similar patch-gap attacks, and we expect the number to increase as more capable open-weight models are released.

Appendix: Safety measures for our experiments +

Notes

1. AI models are becoming very good at decompiling software - turning an executable program back into original source code. By decompiling an upcoming beta, an attacker can still view security fixes before they are applied, making patch-gap attacks possible even when the code is not public. Anthropic demonstrated Mythos was capable of doing this on Windows, but we have not yet tested open weight models. ↵
2. Even if the model creators attempt to add guardrails or train the model to refuse certain requests, by releasing weights publicly, others can create 'abliterated' versions that will comply with any request, and run them without any guardrails. ↵
3. Running Ubuntu 26.04 with default security settings and Firefox downloaded from the official release. ↵
4. 156.0, 156.0.1, 157.0, 157.0.1, 158.0, 158.0.1. Earlier releases are likely also vulnerable, but we did not systematically test prior to this. ↵
5. After finding success with file reads, we tried to achieve arbitrary code execution. We do not have any code execution successes on recent Firefox releases as of writing. ↵
6. Successes required chaining two main vulnerabilities. A patch-gap vulnerability was used to achieve code execution inside the web content process, and the separate, unpublished vulnerability allowed the attack to bypass the sandbox protections that would normally prevent even a compromised renderer process from accessing the filesystem. ↵

About the authors



Aiden Ament

Aiden is a researcher at Epoch AI on the benchmarking team. He previously built enterprise AI security tools at a small startup, focusing on anomaly detection and data protection. He has a BA from UCLA in mathematics and computer science.



Yafah Edelman

Yafah Edelman is the Chief Strategy Officer at Epoch AI. She is interested in understanding and measuring the inputs that allow AI to scale.



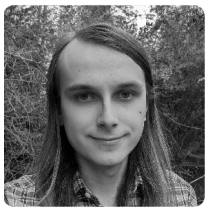
Alexander Barry

Alexander is the head of the Capabilities Tracking team at Epoch AI. He leads our work on the Epoch Capabilities Index, Benchmark Reviews, and RSI tracking. Prior to joining Epoch he worked as a Statistician.



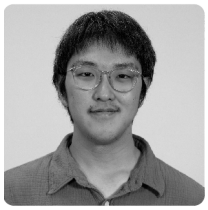
Timothée Chauvin

Timothée is a research engineer at Epoch AI. He has a background in cybersecurity and AI evaluations.



Lee Bousfield

Lee Bousfield is a senior software engineer at Epoch AI, where he works on benchmarking infrastructure and automation. His background is in distributed systems and security engineering.



Jaeho Lee

Jaeho Lee is a researcher at Epoch AI. He has previously worked with the Forecasting Research Institute and Andon Labs, and is currently studying at Brown University.

Related topics

Open models | Capabilities